

DSA

ALGORITHMS

Sorting Algorithms

1. Bubble Sort
 2. Merge Sort
 3. Selection Sort
 4. Insertion Sort
 5. Quick Sort
 6. Heap Sort
 7. Counting Sort
-

Searching Algorithms

1. Linear Search
 2. Binary Search
 3. Jump Search
 4. Interpolation Search
 5. Exponential Search
 6. Ternary Search
-

Dynamic Programming Algorithms

1. Fibonacci Sequence
2. 0/1 Knapsack Problem
3. Longest Common Subsequence (LCS)
4. Longest Increasing Subsequence (LIS)
5. Matrix Chain Multiplication

6. Edit Distance (Levenshtein Distance)

Graph Algorithms

- 1. Depth First Search (DFS)**
 - 2. Breadth First Search (BFS)**
 - 3. Dijkstra's Algorithm**
 - 4. Bellman-Ford Algorithm**
 - 5. Floyd-Warshall Algorithm**
 - 6. A* Search Algorithm**
 - 7. Kruskal's Algorithm**
 - 8. Prim's Algorithm**
 - 9. Tarjan's Algorithm (SCC)**
 - 10. Kosaraju's Algorithm (SCC)**
 - 11. Cycle Detection (Directed & Undirected)**
-

Greedy Algorithms

- 1. Activity Selection Problem**
 - 2. Kruskal's Algorithm**
 - 3. Prim's Algorithm**
-

Backtracking Algorithms

- 1. N-Queens Problem**
 - 2. Sudoku Solver**
-

Mathematical Algorithms

- 1. Euclidean Algorithm (GCD)**
 - 2. Sieve of Eratosthenes**
 - 3. Modular Exponentiation**
-

Tree Algorithms

1. Tree Traversals (In-order, Pre-order, Post-order)
2. Binary Search Tree (BST): Insertion, Deletion, Search
3. Lowest Common Ancestor (LCA)
4. AVL Tree (Self-Balancing BST)

Sorting Algorithms

Searching Algorithms

Graph Algorithms

Dynamic Programming Algorithms

Greedy Algorithms

Backtracking Algorithms

Mathematical Algorithms

Tree Algorithms

Sorting Algorithms

1. Bubble Sort

Theory:

Bubble Sort is a simple comparison-based sorting algorithm. It repeatedly steps through the list, compares adjacent elements, and swaps them if they are in the wrong order. This process is repeated until the list is sorted.

It is called **Bubble Sort** because smaller elements “bubble” to the top of the list, and larger elements sink to the bottom.

Algorithm:

1. Start from the first element, compare it with the next element.
2. If the current element is greater than the next element, swap them.
3. Move to the next element and repeat the process for all elements.
4. Continue the process for all elements until no swaps are needed, meaning the list is sorted.

```

    if (arr[mid] == target) {
        return mid; // Target found
    } else if (arr[mid] < target) {
        low = mid + 1; // Search the right half
    } else {
        high = mid - 1; // Search the left half
    }
}
return -1; // Target not found
}

```

```

public static void main(String[] args) {
    int[] arr = {1, 3, 5, 7, 9, 11};
    int target = 7;
    int result = binarySearch(arr, target);

    if (result == -1) {
        System.out.println("Element not found.");
    } else {
        System.out.println("Element found at index: " + result);
    }
}
}

```

Time Complexity:

- **Best Case:** $O(1)$ (if the element is found at the middle)
- **Average/Worst Case:** $O(\log n)$

3. Merge Sort

Theory:


```
}  
}
```

Time Complexity:

- **Best Case:** $O(n)$ (when the list is already sorted)
 - **Average Case:** $O(n^2)$
 - **Worst Case:** $O(n^2)$
-

2. Binary Search

Theory:

Binary Search is an efficient search algorithm that works on sorted arrays. It repeatedly divides the search interval in half. If the value of the search key is less than the item in the middle of the interval, narrow the interval to the lower half; otherwise, narrow it to the upper half.

Algorithm:

1. Start with two pointers, low and high, representing the current search range.
2. Find the middle element: $mid = (low + high) / 2$.
3. If the middle element equals the target, return the index.
4. If the target is less than the middle element, adjust the high pointer to $mid - 1$.
5. If the target is greater than the middle element, adjust the low pointer to $mid + 1$.
6. Repeat the process until $low > high$.

Code (Java):

java

```
public class BinarySearch {  
    public static int binarySearch(int[] arr, int target) {  
        int low = 0, high = arr.length - 1;  
  
        while (low <= high) {  
            int mid = low + (high - low) / 2; // Avoid overflow
```

```
}  
}
```

Time Complexity:

- **Best Case:** $O(n)$ (when the list is already sorted)
 - **Average Case:** $O(n^2)$
 - **Worst Case:** $O(n^2)$
-

2. Binary Search

Theory:

Binary Search is an efficient search algorithm that works on sorted arrays. It repeatedly divides the search interval in half. If the value of the search key is less than the item in the middle of the interval, narrow the interval to the lower half; otherwise, narrow it to the upper half.

Algorithm:

1. Start with two pointers, low and high, representing the current search range.
2. Find the middle element: $\text{mid} = (\text{low} + \text{high}) / 2$.
3. If the middle element equals the target, return the index.
4. If the target is less than the middle element, adjust the high pointer to $\text{mid} - 1$.
5. If the target is greater than the middle element, adjust the low pointer to $\text{mid} + 1$.
6. Repeat the process until $\text{low} > \text{high}$.

Code (Java):

java

```
public class BinarySearch {  
    public static int binarySearch(int[] arr, int target) {  
        int low = 0, high = arr.length - 1;  
  
        while (low <= high) {  
            int mid = low + (high - low) / 2; // Avoid overflow
```


Code (Java):

java

```
public class BubbleSort {  
    public static void bubbleSort(int[] arr) {  
        int n = arr.length;  
        boolean swapped;  
        for (int i = 0; i < n - 1; i++) {  
            swapped = false;  
            for (int j = 0; j < n - i - 1; j++) {  
                if (arr[j] > arr[j + 1]) {  
                    // Swap arr[j] and arr[j + 1]  
                    int temp = arr[j];  
                    arr[j] = arr[j + 1];  
                    arr[j + 1] = temp;  
                    swapped = true;  
                }  
            }  
            // If no two elements were swapped, break  
            if (!swapped) break;  
        }  
    }  
  
    public static void main(String[] args) {  
        int[] arr = {5, 1, 4, 2, 8};  
        bubbleSort(arr);  
        for (int i : arr) {  
            System.out.print(i + " ");  
        }  
    }  
}
```

Merge Sort is a Divide and Conquer algorithm. It divides the input array into two halves, recursively sorts the two halves, and then merges them to produce the sorted array.

Algorithm:

1. Divide the unsorted array into two halves.
2. Recursively sort both halves.
3. Merge the two halves back together.

Code (Java):

java

```
public class MergeSort {  
    public static void mergeSort(int[] arr, int left, int right) {  
        if (left < right) {  
            int mid = (left + right) / 2;  
  
            // Sort first and second halves  
            mergeSort(arr, left, mid);  
            mergeSort(arr, mid + 1, right);  
  
            // Merge sorted halves  
            merge(arr, left, mid, right);  
        }  
    }  
  
    public static void merge(int[] arr, int left, int mid, int right) {  
        int n1 = mid - left + 1;  
        int n2 = right - mid;  
  
        // Create temporary arrays  
        int[] L = new int[n1];  
        int[] R = new int[n2];
```



```
// Copy data to temp arrays  
for (int i = 0; i < n1; i++) L[i] = arr[left + i];  
for (int j = 0; j < n2; j++) R[j] = arr[mid + 1 + j];
```

```
// Merge the temp arrays
```

```
int i = 0, j = 0;  
int k = left;  
while (i < n1 && j < n2) {  
    if (L[i] <= R[j]) {  
        arr[k] = L[i];  
        i++;  
    } else {  
        arr[k] = R[j];  
        j++;  
    }  
    k++;  
}
```

```
// Copy remaining elements of L[] and R[]
```

```
while (i < n1) {  
    arr[k] = L[i];  
    i++;  
    k++;  
}
```

```
while (j < n2) {  
    arr[k] = R[j];  
    j++;  
    k++;  
}
```

```

    }
}

public static void main(String[] args) {
    int[] arr = {12, 11, 13, 5, 6, 7};
    mergeSort(arr, 0, arr.length - 1);
    for (int i : arr) {
        System.out.print(i + " ");
    }
}
}

```

Time Complexity:

- **Best Case:** $O(n \log n)$
- **Average Case:** $O(n \log n)$
- **Worst Case:** $O(n \log n)$

4. Fibonacci Sequence (Dynamic Programming)

Theory:

The Fibonacci sequence is a series of numbers where each number is the sum of the two preceding ones. The dynamic programming approach avoids recalculating Fibonacci values by storing the results of subproblems.

Algorithm:

1. Use an array to store the Fibonacci values.
2. Start with base cases: $\text{Fib}(0) = 0$, $\text{Fib}(1) = 1$.
3. For each next value, calculate $\text{Fib}(n) = \text{Fib}(n-1) + \text{Fib}(n-2)$ and store it.

Code (Java):

```
java
```

```

public class FibonacciDP {
    public static int fibonacci(int n) {

```



```

    if (n <= 1) {
        return n;
    }

    int[] fib = new int[n + 1];
    fib[0] = 0;
    fib[1] = 1;

    for (int i = 2; i <= n; i++) {
        fib[i] = fib[i - 1] + fib[i - 2];
    }

    return fib[n];
}

public static void main(String[] args) {
    int n = 10;
    System.out.println("Fibonacci of " + n + " is: " + fibonacci(n));
}
}

```

Time Complexity:

- Time Complexity: $O(n)$
- Space Complexity: $O(n)$

5. Selection Sort

Theory:

Selection Sort is a simple comparison-based sorting algorithm. It repeatedly selects the smallest element from the unsorted portion of the array and swaps it with the first unsorted element. This process continues until the entire array is sorted.